# AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

**Dept. of Computer Science**
**Faculty of Science and Technology**

## CSC2210: OBJECT ORIENTED PROGRAMMING 2

**Fall 2025-2026**

**Section: Z**

**Group No: 0**

# Project Report On

# "Pharmacy Management System"

**Supervised By**

**Kazi Sadia**

**Submitted By:**

| Name | ID |
|---|---|
| 1. MD. NYEEMUL ISLAM | 23-50954-1 |

| **CO2:** Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application. | | | | | |
|---|---|---|---|---|---|
| **Assessment Criteria** | **Not Attended/ Incorrect (0)** | **Inadequate (1-2)** | **Average (3)** | **Good (4)** | **Excellent (5)** |
| **Evaluation Criteria** | **Evaluation Definition** | | | | **Total =** |
| Requirement fulfillment | Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities. | | | | |
| Validation | Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data. | | | | |
| Verification | Identifying if the students can verify the system data along with proper functional requirements in terms of data flow. | | | | |

Table of Contents: Page no.

1. Chapter :01 (Introduction)---------- 03
2. Chapter :02(User Story)---------- 03
3. Chapter :03 (ER Diagram)---------- 05
4. Chapter :04 (SQL Queries)---------- 06
5. Chapter:05 (Screenshots) ---------- 07

# Chapter :01 (Introduction)

The **Pharmacy Management System** is a database-driven, role-based software application developed to streamline and automate the daily operations of a pharmacy, including user management, medicine handling, inventory control, sales processing, and report generation. The system is designed to reduce manual workload, minimize operational errors, and improve overall efficiency by managing all pharmacy-related activities through a centralized digital database.

This application enables **Administrators, Pharmacists, and Customers** to perform their respective tasks through role-based access control. Each user role is granted specific permissions to ensure secure and efficient operation of the system. The system manages all core pharmacy functions—from maintaining medicine records and tracking stock levels to processing sales transactions, generating bills, and maintaining user and customer information—within a single integrated platform.

Key components of the system include:

- Role Management for defining and controlling access levels of Admins, Pharmacists, and Customers.
- User Management for adding, updating, and maintaining administrator and pharmacist account information.
- Customer Management for storing customer details, purchase history, and billing records.
- Medicine Management to add, update, categorize, and maintain medicine information including pricing and expiry dates.
- Settings Module allowing users to update personal details and change passwords securely based on their role.

The system is built using **C# (Windows Forms)** and **SQL Server**, providing a normalized relational database that ensures data consistency, integrity, and fast query performance. It serves as a robust and user-friendly foundation for hotels to deliver organized, efficient, and professional management services to their customers.

# Chapter :02 (User Story)

## Customer User Stories

**1. As a Customer,**
I want to search and select medicines based on availability, So that I can purchase the required medicines when needed.

- The system retrieves medicine details from the Medicines table and checks stock availability from the Inventory table.

- Selected medicine details (medicine ID, quantity, price) are added to a temporary Sales/Cart record.

**2. As a Customer,**
I want to make a payment at the time of purchase or later, So that I can complete my purchase according to my convenience.

- The system records payment details in the **orders** table linked to my **userID** and **orderID**.
- Payment method is stored as **Cash** or **Card**, and payment status is recorded as **Unpaid** or **Paid**.

**3. As a Customer,**
I want to view my purchase history and billing records, So that I can track my previous purchases and payments.

- The system retrieves order records from the **orders** filtered by my **userID**.
- Each record displays medicine name, quantity, purchase , order status, total amount, and payment status.

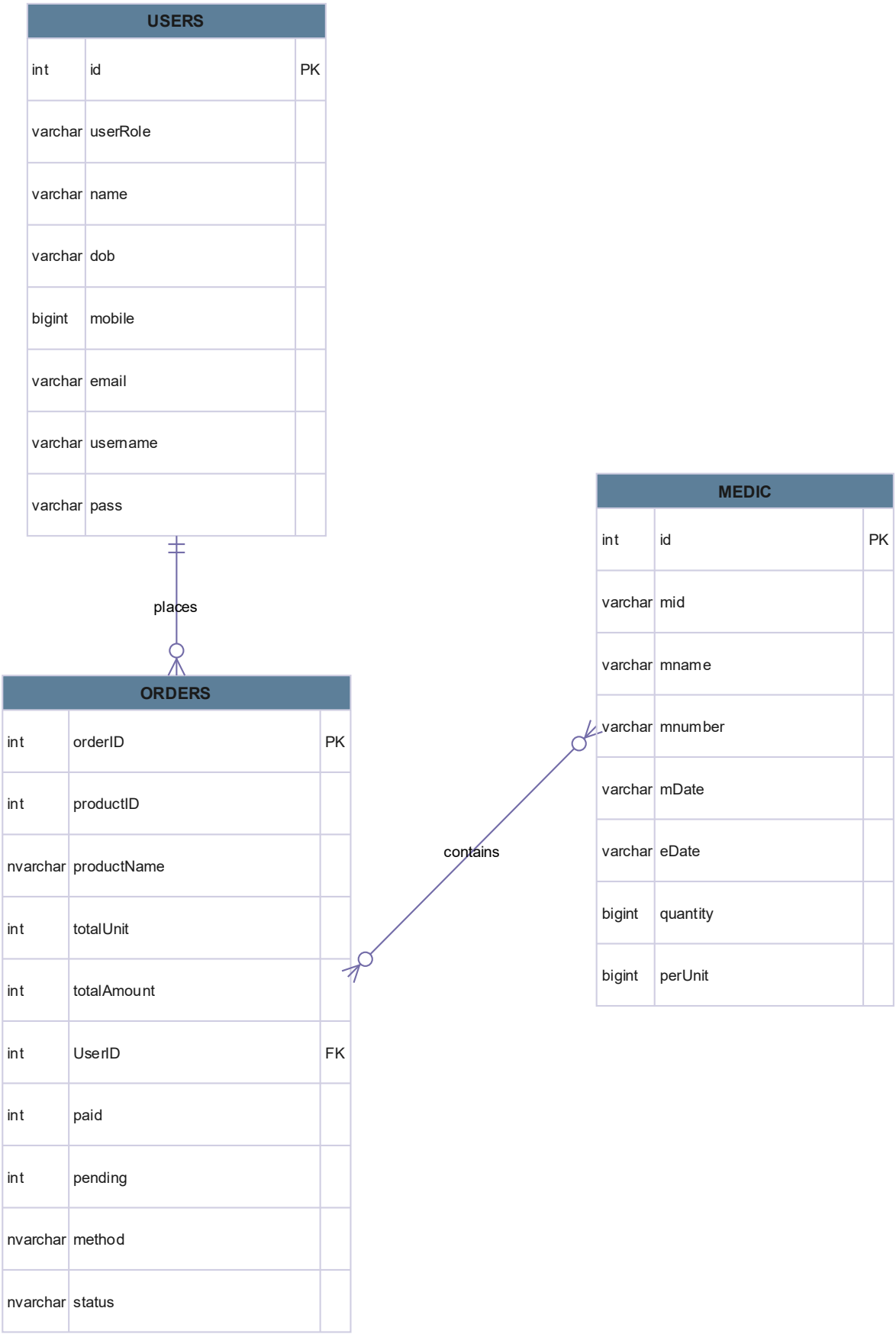
**4. As a Customer,**
I want a dashboard overview, So that I can monitor my total purchases, total spending, and outstanding payments.

- The system calculates and displays total orders, total ordered units, total amount spent, and unpaid balance.
- All information is presented in a clear and concise dashboard view.

**5. As a Customer,**
I want to create a new account,
So that I can access the system, purchase medicines, and view my information.

- The system collects required details such as Full Name, Email, Phone,Username, Password.
- The system validates that Email and Phone are unique and stores the information in the **Users** table with the default Customer role.

# Chapter :03 (ER Diagram)



# Chapter :04 (SQL Queries)

1. $"select count(*) 'orderID' from orders where orders.UserID='{id}'; \r\n" +
   $"select sum(totalUnit) 'Units' from orders where orders.UserID = '{id}'; \r\n" +
   $"select sum(paid) 'Spending' from orders where orders.UserID = '{id}'; \r\n" +
   $"select sum(pending) 'Unpaid' from orders where orders.UserID = '{id}';";

2. $"select * from medic";

3. $"insert into orders
   (productID,productName,totalUnit,totalAmount,UserID,paid, pending,method,status)
   values('{productID}','{productName}','{unit}','{totalAmount}','{userID}','{totalAmount}','0','{method}','Paid'); " +
   $"UPDATE medic SET quantity = quantity - {int.Parse(unit)} WHERE id = '{productID}'";

4. $"update Users set name='{fullname}',mobile='{phone}', email='{email}' Where id='{id}'";

5. $"select Password from Users where id='{id}'";

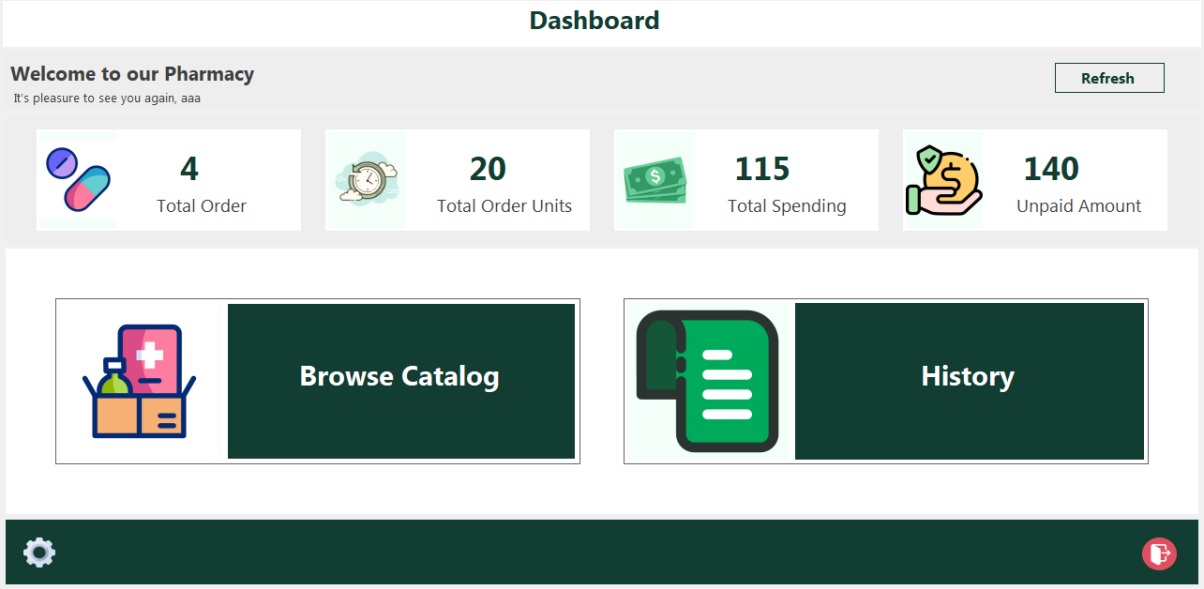
6. $"insert into Users
   (userRole,name,mobile,email,username,pass)values('{role}','{fullname}','{phone}','{email}','{username}','{password}')";

7. $"insert into orders
   (productID,productName,totalUnit,totalAmount,UserID,paid,pending,method,status)
   values('{productID}','{productName}','{unit}','{totalAmount}','{userID}','0','{totalAmount}','Cash','Unpaid'); " +
   $"UPDATE medic SET quantity = quantity - {int.Parse(unit)} WHERE id = '{productID}'";

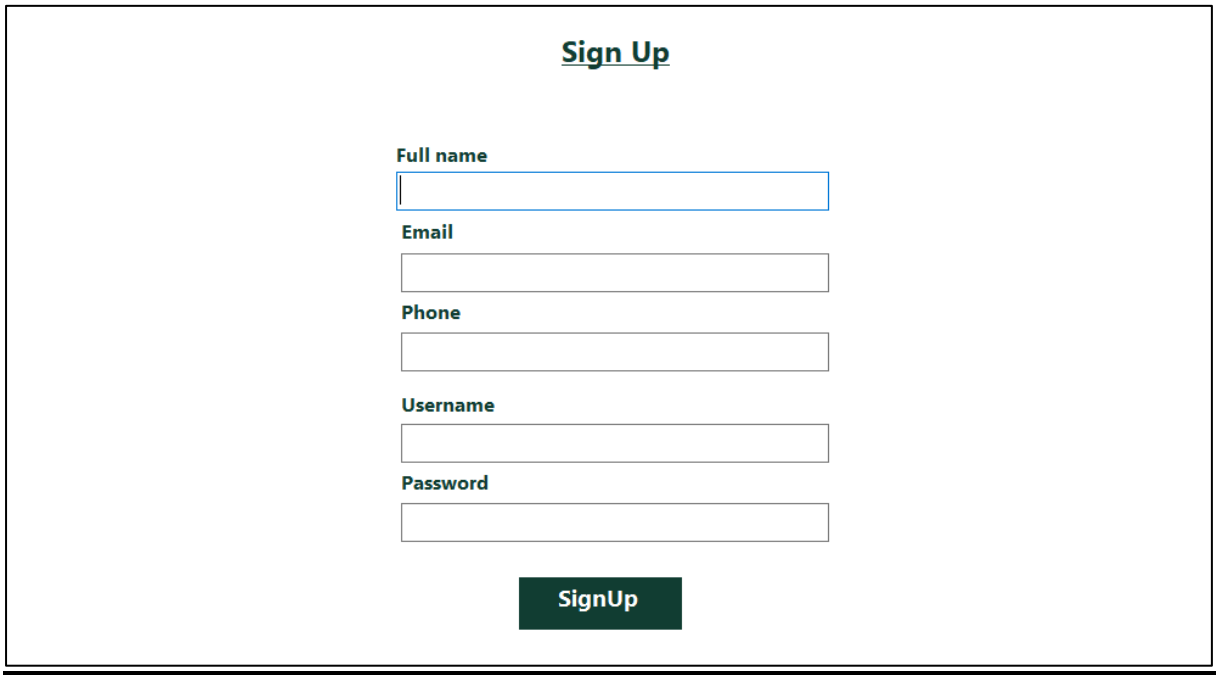
| orderID | productID | productName | totalUnit | totalAmount | UserID | paid | pending | method | status |
|---|---|---|---|---|---|---|---|---|---|
| 14 | 7 | Metformin | 4 | 60 | 7 | 35 | 60 | Cash | Paid |
| 15 | 4 | Vitamin | 5 | 35 | 7 | 0 | 35 | Cash | Unpaid |
| 16 | 6 | Calcium | 8 | 80 | 7 | 80 | 0 | Cash | Paid |
| 17 | 7 | Metformin | 3 | 45 | 7 | 0 | 45 | Cash | Unpaid |
| NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL | NULL |

**Table: orders**

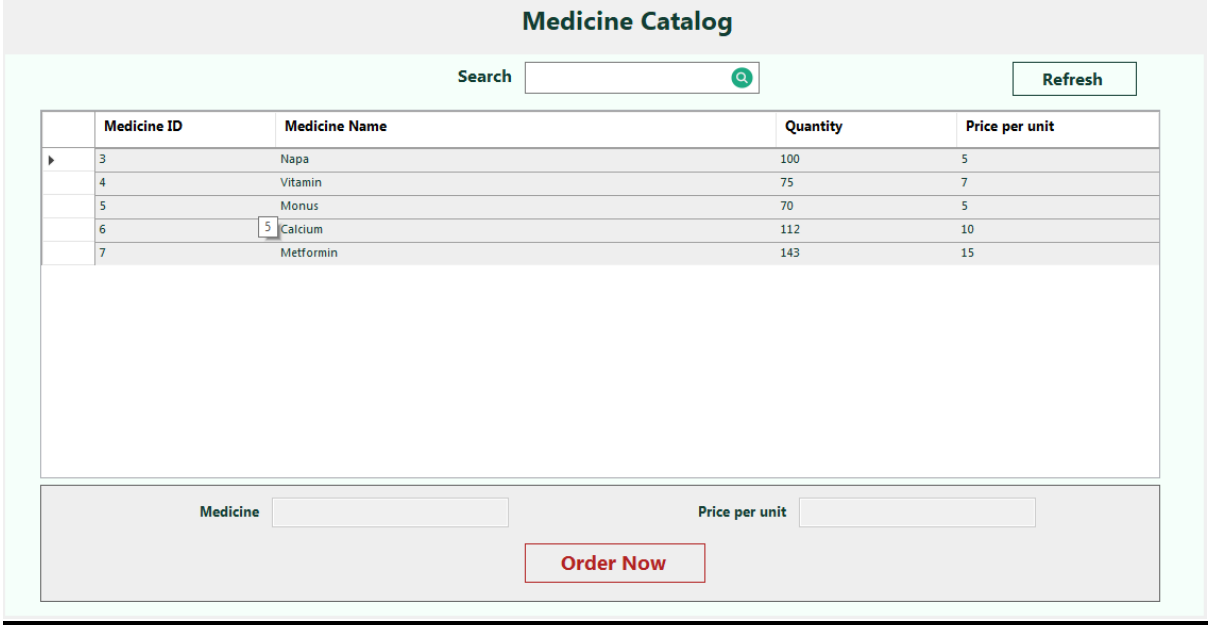
# Chapter :05 (Screenshot)




**Customer Dashboard**

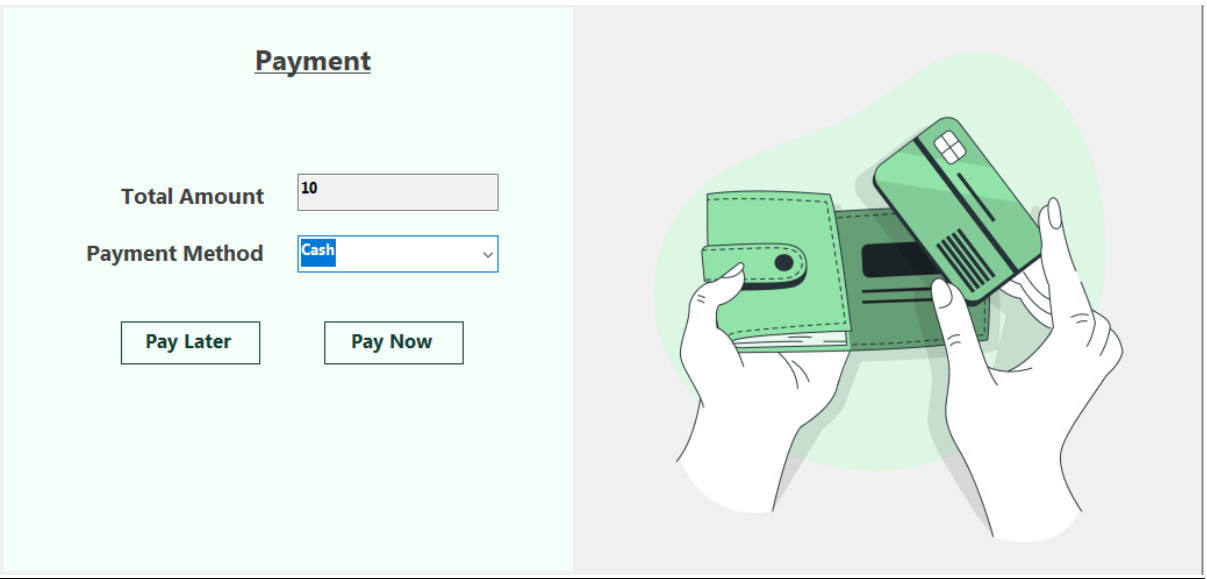



**Customer SignUp**

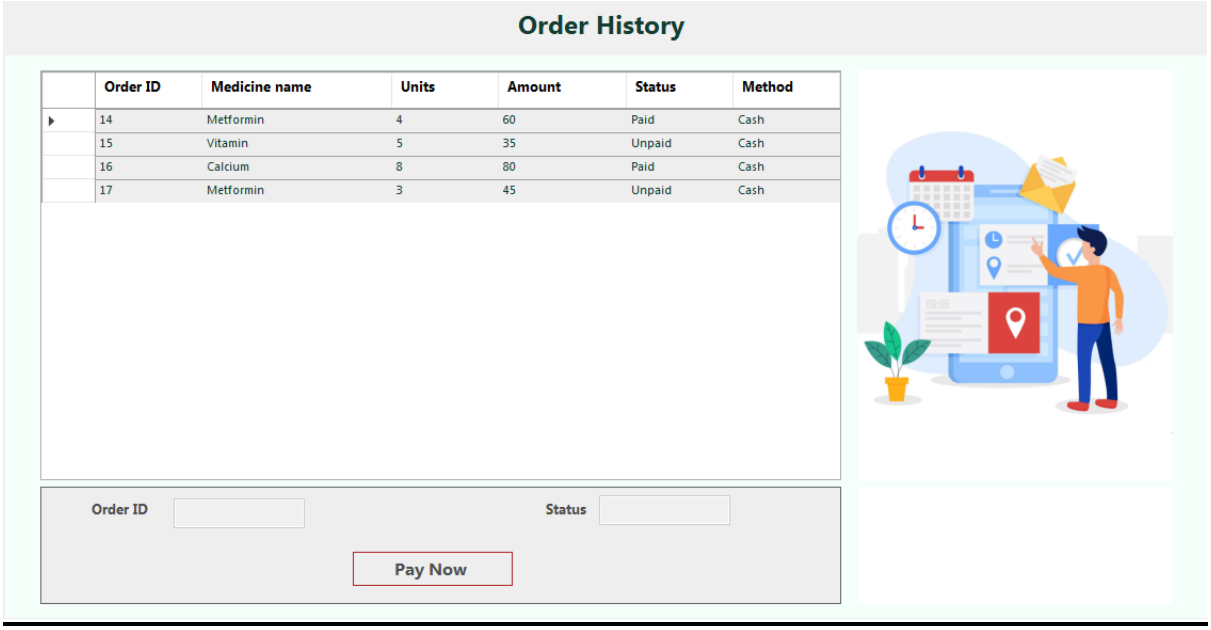



**Medicine Catalog**

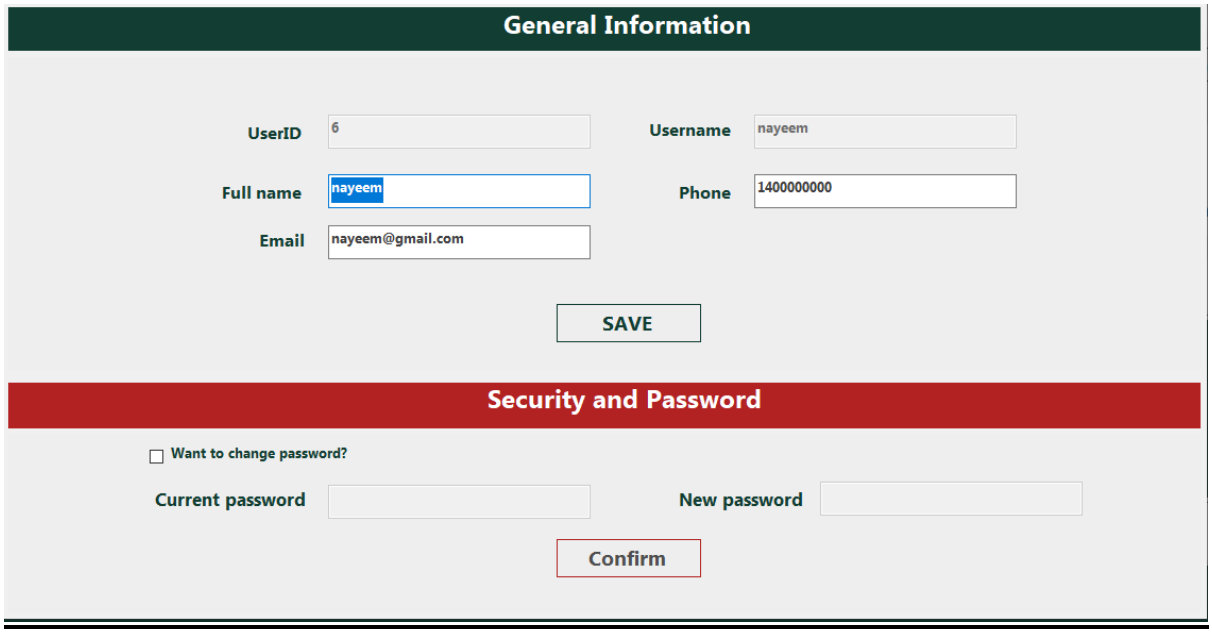
**Order Details**




**Payment Details**




**Customer History**




## Customer Settings